

Orchestrator Logic Guide

Health Sciences Industry Solutions Architect

A detailed reference for code owners, reviewers, and contributors.

Repository: Snowflake-Solutions/health-sciences-coco-skills-incubator

Generated from: agents/health-sciences-incubator.md

Table of Contents

- 1. Overview -- What the Orchestrator Does
- 2. Architecture -- How It's Built
- 3. Plan-then-Execute Protocol -- The Mandatory Gate
- 4. Platform Affinities -- Automatic Platform Skill Selection
- 5. Routing Logic -- How Requests Reach Skills
- 6. Skill Taxonomy -- The 5-Level Hierarchy
- 7. Cross-Domain Composition Patterns
- 8. Cortex Knowledge Extensions (CKEs)
- 9. Guardrails and Anti-Patterns
- 10. Generation Pipeline -- Template to Agent
- 11. QA Validation -- 12-Check Suite
- 12. File Reference

1. Overview -- What the Orchestrator Does

The orchestrator is a system prompt (Markdown file) that is loaded into Cortex Code when a user activates the health-sciences-incubator profile. It acts as the "brain" of the Health Sciences Industry Solutions Architect -- receiving natural language requests from users and composing the right combination of industry skills and Snowflake platform capabilities into end-to-end solutions.

It does NOT execute code itself. Instead, it:

- Detects the healthcare domain from the user's request (Provider, Pharma, Payer)
- Routes to one or more industry skills using trigger-keyword matching
- Checks platform affinities to add Snowflake platform skills (governance, Streamlit, ML, etc.)
- Builds a numbered solution plan and presents it for user approval
- Executes the approved plan step-by-step, invoking skills via the `skill` tool
- Applies HIPAA governance guardrails as cross-cutting concerns

Key Design Principles

- Composable: Skills are independent building blocks, not monolithic scripts
- Plan-gated: No execution without explicit user approval
- Template-driven: Generated from YAML registry + Jinja2 template (not hand-edited)
- Knowledge-grounded: CKEs provide RAG search over PubMed and ClinicalTrials.gov
- Governance by default: HIPAA guardrails enforced across all workflows

2. Architecture -- How It's Built

Three-Layer Architecture

The system has three layers that work together:

Layer 1: YAML Registry (templates/skills_incubator.yaml)

The single source of truth for all skills. Each skill entry includes: name, triggers (keywords for routing), description, domain (taxonomy placement), and optional metadata (sub_skills, cke, used_by, standalone). The registry also defines cross-domain composition patterns, overlaps between skills, and profile metadata (name, description, intro text).

Layer 2: Jinja2 Template (templates/orchestrator.md.j2)

A parameterized Markdown template that defines the orchestrator's structure: Plan-then-Execute protocol, Platform Skill Selection, routing rules, skill taxonomy, routing tables, cross-domain patterns, guardrails, and anti-patterns. The template reads from the registry to populate skill-specific sections (taxonomy tree, routing tables, CKE routing, patterns). Structural sections (protocol, guardrails, routing rules) are static in the template.

Layer 3: Generated Agent (agents/health-sciences-incubator.md)

The output -- a complete Markdown file with YAML frontmatter (name, description, tools) that Cortex Code loads as a system prompt. This file should NEVER be hand-edited; all changes go through the registry or template, then regeneration.

Generation Flow

```
skills_incubator.yaml --+
                        |--> generate_orchestrators.py --> health-sciences-incubator.md
orchestrator.md.j2      --+
```

Run: `python scripts/generate_orchestrators.py --profile incubator`

Twin Orchestrator Model

Two orchestrators are generated from the same template but different registries:

Property	Incubator	Production
Registry	skills_incubator.yaml	skills_production.yaml
Output	health-sciences-incubator.md	health-sciences-solutions.md
Skills	All skills (experimental + mature)	Only graduated skills
Audience	SEs, SAs, contributors	Field teams, customers
Gate	Skills land here via PR to main	Skills graduate via Tiger Team review

3. Plan-then-Execute Protocol

Every health sciences task follows a mandatory two-phase protocol. Phase 1 (Plan) MUST complete before Phase 2 (Execute) can begin. This is non-negotiable.

Phase 1: Plan (Mandatory Gate)

The orchestrator performs these steps to build a plan:

1. Identify the sub-industry (Provider, Pharma, Payer) from the Routing Rules
2. Route by task if sub-industry is ambiguous
3. Scan the Skill Routing Tables for trigger keyword matches
4. Check Cross-Domain Patterns if the request spans multiple business functions
5. Check Platform Affinities for each skill in the plan -- add platform skills where conditions match
6. Build a numbered solution plan. Each step specifies: skill name, what it produces, dependencies, and governance applicability
7. Present the plan to the user via ask_user_question and wait for explicit approval

Phase 2: Execute (Only After Approval)

1. Execute each step in the approved plan order
2. Invoke skills using the skill tool -- never bypass skills with raw SQL/Bash
3. Run preflight checks (CKEs, Data Model Knowledge auto-detect availability)
4. Apply governance guardrails on all patient/clinical data
5. Enrich with CKEs when the plan calls for evidence grounding
6. Report back after each major step for user course-correction
7. Test and validate before declaring success

When the Gate Is Lightweight

The plan gate can be a single sentence + confirmation for:

- Simple single-skill queries (e.g., 'What adverse events are associated with aspirin?')
- Informational questions (e.g., 'What skills are available for genomics?')
- Follow-up steps within an already-approved plan

For everything else -- multi-step pipelines, cross-domain composition, anything touching patient data -- the full plan gate is mandatory.

Example: Plan Gate in Action

User: Design a Phase III trial for a GLP-1 receptor agonist for T2D

Orchestrator builds plan:

1. \$hcls-cross-cke-clinical-trials -> search for STEP trial references
2. \$hcls-cross-cke-pubmed -> review STEP trial publications
3. \$hcls-pharma-genomics-survival-analysis -> power analysis for endpoints
4. \$hcls-pharma-dsafety-clinical-trial-protocol -> generate protocol
5. cortex-ai-functions -> AI_COMPLETE for narrative generation (affinity)

Orchestrator presents plan via ask_user_question -> user approves -> execute

4. Platform Affinities

Platform affinities are a declarative mechanism for industry skills to declare which Snowflake platform skills enhance them and under what conditions. During Phase 1 (Plan), the orchestrator reads each skill's affinities and evaluates conditions against the user's request to automatically sequence platform skills into the solution plan.

How It Works

Each SKILL.md declares platform_affinities in its YAML frontmatter:

```
---
name: hcsl-provider-cdata-fhir
platform_affinities:
  produces: [tables, views, stages]
  benefits_from:
    - skill: dynamic-tables
      when: "incremental refresh or ongoing FHIR feeds"
    - skill: data-governance
      when: "FHIR tables contain PHI"
    - skill: developing-with-streamlit
      when: "user wants a patient data dashboard"
---
```

Affinity Evaluation Algorithm

1. For each domain skill in the plan, read its platform_affinities from SKILL.md frontmatter
2. For each benefits_from entry, evaluate the 'when' condition against the user's request
3. If the condition matches, add that platform skill as a follow-on step in the plan
4. Deduplicate: if multiple skills trigger the same platform skill, include it once

produces vs benefits_from

Field	Purpose	Example
produces	Snowflake objects this skill creates	tables, views, stages, cortex_search_service
benefits_from	Platform skills that enhance this skill + conditions	data-governance when 'PHI present'

10 Platform Skills Available

Platform Skill	When to Include
dynamic-tables	Incremental refresh, ongoing data feeds, streaming pipelines
data-governance	PHI/PII present, masking policies, row-access policies, audit
data-quality	Data validation, conformance checks, completeness monitoring
semantic-view	Natural language queries, analytics layer, BI integration
developing-with-streamlit	Dashboards, viewers, interactive UIs
deploy-to-spcs	Container services, GPU compute, custom viewers
machine-learning	Model training, registry, deployment, inference
cortex-ai-functions	AI_PARSE_DOCUMENT, AI_COMPLETE, AI_EXTRACT, text analytics

cortex-agent	Conversational agents over domain data
search-optimization	Full-text or semantic search over extracted content

Worked Example

User: "Build a FHIR data pipeline with a patient dashboard and PHI masking"

1. \$hcsls-provider-cdata-fhir selected (triggers: FHIR, HL7, Patient resource)
2. Read affinities: produces=[tables, views, stages]
3. Evaluate: dynamic-tables when 'incremental refresh' -> YES (pipeline = ongoing feeds)
4. Evaluate: data-governance when 'PHI present' -> YES (user said PHI masking)
5. Evaluate: developing-with-streamlit when 'dashboard' -> YES
6. Final plan: FHIR ingest -> Dynamic Tables -> Data Governance -> Streamlit dashboard

5. Routing Logic

The orchestrator uses a four-step routing algorithm to determine which skills to invoke. Routing happens during Phase 1 (Plan) and determines the skill composition.

Step 1: Route by Sub-Industry

Determine the customer/context type:

Customer Type	Sub-Industry	Examples
Hospital, health system, clinic, IDN	Provider	Epic, Cerner, clinical research orgs
Pharma, biotech, CRO	Pharma	Drug development, trials, genomics
Health plan, TPA, PBM	Payer	Claims adjudication, member analytics

Step 2: Route by Task (Disambiguation)

When the customer straddles sub-industries (e.g., CRO doing hospital-based trials), route by the TASK being performed, not the customer type.

Task Type	Route To	Regardless Of
Clinical data / EHR	Provider > Clinical Data Mgmt	Customer type
Drug safety / adverse events	Pharma > Drug Safety	Customer type
Imaging workflows	Provider > Clinical Research	Customer type
Genomic analysis	Pharma > Genomics	Customer type
Claims analysis	Provider > Revenue Cycle	Until Payer skills exist

Step 3: Cross-Industry Skills

These skills are available to ALL sub-industries. The orchestrator invokes them whenever they add value, regardless of routing path:

- hcls-cross-research-problem-selection -- scientific problem validation
- hcls-cross-skill-development -- contributor workflow to add new skills
- hcls-cross-cke-pubmed -- PubMed biomedical literature search
- hcls-cross-cke-clinical-trials -- ClinicalTrials.gov registry search

Step 4: Accept Overlaps

Some skills serve multiple sub-industries. The orchestrator routes to them regardless of which tree they sit in:

- claims-data-analysis -- Provider (revenue cycle) + Payer (claims processing)
- survival-analysis -- Pharma (clinical outcomes) + Provider (clinical research)
- clinical-nlp -- Provider (EHR extraction) + Pharma (safety narrative mining)
- clinical-docs -- Provider (document intelligence) + Pharma (safety narrative extraction)

Skill-First Rule

Always check skills before using raw tools. If a matching skill exists, invoke it as the FIRST action. Skills encode domain

expertise, gated workflows, guardrails, and best practices that raw tool usage (SQL, Bash) does not provide.

6. Skill Taxonomy

Skills are organized in a five-level hierarchy:

Industry / Sub-Industry / Business Function / Use Case Skill / Sub-Skill

The naming convention encodes this hierarchy in a flat directory structure:

hcls-{sub-industry}-{function}-{skill}

Skill Types

Type	Description	Example
Router	Detects intent, routes to sub-skills. Has setup, preflight, workflow.	hcls-provider-imaging (7 sub-skills)
Sub-skill	Handles one task within a router. Loaded by router, not user.	dicom-parser, clinical-docs-search
Standalone	Self-contained, no router or sub-skills.	hcls-provider-cdata-fhir

Full Taxonomy Tree

```

Health Sciences
|-- Provider
|   |-- Clinical Research
|   |   |-- hcls-provider-imaging (router + 7 sub-skills)
|   |   +-- hcls-provider-imaging-dicom-parser (standalone)
|   |-- Clinical Data Management
|   |   |-- hcls-provider-cdata-fhir
|   |   |-- hcls-provider-cdata-clinical-nlp
|   |   |-- hcls-provider-cdata-omop
|   |   +-- hcls-provider-cdata-clinical-docs (router + 5 sub-skills)
|   +-- Revenue Cycle
|       +-- hcls-provider-claims-data-analysis
|
|-- Pharma
|   |-- Drug Safety
|   |   |-- hcls-pharma-dsafety-pharmacovigilance
|   |   +-- hcls-pharma-dsafety-clinical-trial-protocol
|   |-- Genomics
|   |   |-- hcls-pharma-genomics-nextflow
|   |   |-- hcls-pharma-genomics-variant-annotation
|   |   |-- hcls-pharma-genomics-single-cell-qc
|   |   |-- hcls-pharma-genomics-scvi-tools
|   |   +-- hcls-pharma-genomics-survival-analysis
|   +-- Lab Operations
|       +-- hcls-pharma-lab-allotrope
|
+-- Cross-Industry
    |-- Research Strategy: hcls-cross-research-problem-selection
    |-- Skill Development: hcls-cross-skill-development
    +-- Knowledge Extensions: cke-pubmed, cke-clinical-trials
  
```

Domain Mapping (Registry -> Orchestrator)

Each skill in `skills_incubator.yaml` has a 'domain' field that maps to a section in the generated orchestrator. The `DOMAIN_ORDER` list in `generate_orchestrators.py` controls the section ordering:

- Provider > Clinical Research
- Provider > Clinical Data Management
- Provider > Revenue Cycle
- Pharma > Drug Safety
- Pharma > Genomics
- Pharma > Lab Operations
- Cross-Industry > Research Strategy
- Cross-Industry > Skill Development
- Cross-Industry > Knowledge Extensions

7. Cross-Domain Composition Patterns

When a user's request spans multiple business functions, the orchestrator composes skills using predefined patterns. Patterns are guides, not rigid scripts -- the orchestrator adapts them to the user's actual request.

Pattern	Skill Chain
Imaging + Clinical Integration	DICOM parse -> FHIR ingest -> Clinical NLP -> PubMed enrichment -> UI
Clinical Data Warehouse (OMOP)	FHIR ingest -> OMOP CDM transform -> HIPAA governance -> Semantic views
Drug Safety Signal Detection	FAERS analysis -> PubMed literature -> Clinical NLP -> Claims correlation
Genomics + Clinical Outcomes	nf-core pipeline -> Variant annotation -> Survival analysis -> ML models
Single-Cell Analysis Pipeline	scRNA-seq QC -> scvi-tools integration -> ML Registry
Real-World Evidence Study	Claims cohort -> ClinicalTrials.gov -> OMOP -> Survival -> PubMed validation
Clinical Trial Design	Problem validation -> Trial search -> Literature -> Protocol -> Power analysis
Lab Data Modernization	Allotrope conversion -> Dynamic Tables -> Analytics dashboard
Clinical Data Application (React)	Domain skills -> React/Next.js app -> SPCS deployment -> PHI masking
Clinical Document Intelligence	Clinical docs extraction -> NLP enrichment -> PubMed -> FHIR -> Governance -> Semantic views

Adapting Patterns

- Skip steps that don't apply
- Reorder when the user already has intermediate outputs
- Combine patterns when the request spans multiple
- Add steps for capabilities not in the pattern (e.g., governance)
- Always ask if the adaptation is unclear

8. Cortex Knowledge Extensions (CKEs)

CKEs are standalone composable skills backed by Cortex Search Services from Snowflake Marketplace. They provide on-demand RAG search over external knowledge corpora. Domain skills invoke them when evidence grounding adds value.

Available CKEs

CKE Skill	Data Source	Used By
hcls-cross-cke-pubmed	PubMed biomedical literature	pharmacovigilance, clinical-nlp, research-problem-selection, imaging (analytics), clinical-docs
hcls-cross-cke-clinical-trials	ClinicalTrials.gov registry	clinical-trial-protocol, claims-data-analysis, survival-analysis

Preflight Pattern

Before invoking any CKE, the skill runs a probe query to verify the Marketplace listing is installed. If MISSING, the skill skips CKE enrichment gracefully and continues with its primary task. This ensures skills work without CKEs but provide richer results with them.

Data Model Knowledge (Internal CKE)

In addition to external CKEs, two router skills (imaging, clinical-docs) use internal CKE layers -- Cortex Search Services over their own data models. These auto-fire as a pre-step (Step 0) to ground DDL generation, extraction config, and schema queries in live reference models.

Router	Search Service	What It Answers
hcls-provider-imaging	DICOM_MODEL_SEARCH_SVC	Table definitions, column types, DICOM tags, PHI indicators
hcls-provider-cdata-clinical-docs	CLINICAL_DOCS_MODEL_SEARCH_SVC + CLINICAL_DOCS_SPECS_SEARCH_SVC	Schema + doc type specs, extraction prompts, field definitions

9. Guardrails and Anti-Patterns

HIPAA Guardrails (Always Enforced)

- Always apply HIPAA governance before exposing any patient data
- Never store or display PHI without masking policies in place
- Always use `IS_ROLE_IN_SESSION()` (not `CURRENT_ROLE()`) in masking/row-access policies
- Always recommend audit trails via `ACCESS_HISTORY` for PHI-containing tables
- Prefer de-identified datasets for analytics and ML training
- Always validate FHIR/HL7/OMOP data quality before building downstream tables
- For genomic data: ensure proper consent tracking and data use agreements
- For FAERS/pharmacovigilance: note limitations of spontaneous reporting data

Anti-Patterns (Do NOT)

- Do NOT use clinical-nlp on raw files (PDF, DOCX, images) -- use clinical-docs first
- Do NOT use survival-analysis without a defined cohort -- build the cohort first
- Do NOT invoke CKEs for non-evidence tasks (pipeline construction, SQL generation)
- Do NOT skip preflight checks -- they run automatically
- Do NOT force-follow a pattern when the request only partially matches -- adapt it
- Do NOT use imaging-dicom-parser (standalone) for full imaging workflows -- use the router
- Do NOT bypass the plan gate for multi-step pipelines or patient data workflows

10. Generation Pipeline

The orchestrator is never hand-edited. All changes flow through a generation pipeline:

Pipeline Steps

1. Edit the YAML registry (templates/skills_incubator.yaml) to add/modify skills, patterns, overlaps, or CKE metadata
2. Edit the Jinja2 template (templates/orchestrator.md.j2) to change structural sections (protocol, guardrails, routing rules)
3. Run: `python scripts/generate_orchestrators.py --profile incubator`
4. The script loads the YAML, builds SkillObj instances, groups by domain, filters patterns, and renders the template
5. Output is written to agents/health-sciences-incubator.md
6. If `--profile both`, a drift check compares the two orchestrators and flags unexpected structural differences

What Goes Where

Change Type	Edit In
Add a new skill	skills_incubator.yaml (skills section)
Add triggers or description	skills_incubator.yaml (skill entry)
Add a cross-domain pattern	skills_incubator.yaml (patterns section)
Add an overlap	skills_incubator.yaml (overlaps section)
Change routing rules	orchestrator.md.j2 (Routing Rules section)
Change Plan-then-Execute protocol	orchestrator.md.j2 (protocol section)
Change guardrails or anti-patterns	orchestrator.md.j2 (guardrails section)
Add platform affinities	SKILL.md frontmatter (in skill directory)
Change Platform Skill Selection logic	orchestrator.md.j2 (Platform section)

Key Code: generate_orchestrators.py

The generator script is ~190 lines of Python. Key components:

- SkillObj class: wraps each skill's YAML data (name, triggers, description, domain, sub_skills, cke, etc.)
- build_skills(): creates OrderedDict of SkillObj from registry
- build_skills_by_domain(): groups skills by domain in DOMAIN_ORDER sequence
- filter_patterns(): removes patterns with unavailable skills (for production)
- render(): loads Jinja2 template, passes all data, returns rendered Markdown
- Drift check: after generating both profiles, compares line-by-line and flags unexpected structural differences beyond skill refs and profile metadata

11. QA Validation -- 12-Check Suite

The QA script (scripts/qa_validate_orchestrator.py) validates the orchestrator against the filesystem and registry. Run it before every commit:

```
python scripts/qa_validate_orchestrator.py
```

Check	Name	What It Validates
1	\$refs -> SKILL.md name	Every \$ref in orchestrator points to a SKILL.md with matching name
2	SKILL.md -> orchestrator ref	Every top-level SKILL.md is referenced in the orchestrator
3	Folder name == SKILL.md name	Directory name matches the name: field in SKILL.md frontmatter
4	Imaging sub-skills	All imaging sub-skill directories exist and are referenced
5	Taxonomy tree entries	Skills in the taxonomy tree exist in the filesystem
6	Reference consistency	Counts \$ref occurrences per skill (informational)
7	Standalone skills	Standalone skills (e.g., dicom-parser) exist and are referenced
8	Twin drift	Structural differences between incubator and production orchestrators
9	Registry bidirectional	Every registry entry has a directory and vice versa
10	Platform affinities	All SKILL.md files have valid platform_affinities frontmatter
11	CKE used_by	CKE used_by references point to real skills
12	Overlap entries	Overlap skills exist and are referenced in orchestrator

Expected results: All checks pass except CHECK 8 (twin drift) which shows expected structural differences because the production orchestrator is still a scaffold with no graduated skills.

12. File Reference

File	Purpose
agents/health-sciences-incubator.md	Generated orchestrator (system prompt) -- NEVER hand-edit
agents/health-sciences-solutions.md	Generated production orchestrator -- NEVER hand-edit
templates/orchestrator.md.j2	Jinja2 template for orchestrator structure
templates/skills_incubator.yaml	YAML registry -- source of truth for all skills
templates/skills_production.yaml	YAML registry -- production skills only
scripts/generate_orchestrators.py	Generation script: YAML + Jinja2 -> .md
scripts/qa_validate_orchestrator.py	12-check QA validation suite
skills/hcls-*/SKILL.md	Industry skill definitions with platform_affinities

End of Document